机器学习
第 5 章　KNN 算法

——最简单的算法：近朱者赤，近墨者黑

2026 年 3 月 17 日

本章题目导览：你将挑战哪些核心问题？

KNN（K-Nearest Neighbors）是机器学习中最直观、最易理解的算法之一——”你的 k 个最近邻是什么类别，你就是什么类别”。本章 5 道题覆盖 KNN 的完整体系：

- **Q5-01（原理推导）**：KNN 工作原理手动计算——给定数据点，算距离，找邻居，投票决策。
- **Q5-02（参数分析）**：k 值如何影响决策边界？过大/过小的 k 分别导致什么问题？
- **Q5-03（距离度量）**：欧氏距离、曼哈顿距离、闵可夫斯基距离的计算与比较。
- **Q5-04（API 实践）**：sklearn KNN 分类与回归的参数解析与代码实现。
- **Q5-05（案例综合）**：心脏病预测——完整的 KNN 建模流程，包括特征工程、超参数调优。

1　第 5 章　KNN：最直观的机器学习算法

先建立一个总感觉：KNN 在做什么？

KNN 的逻辑极其简单，就是现实生活中的一句话：”**近朱者赤，近墨者黑。**”

如果你不知道一个人是好人还是坏人，就看他最亲近的 k 个朋友——如果大部分朋友是好人，那他大概率也是好人。

KNN 没有显式的”训练过程”：它把整个训练集”记住”，预测时直接找最近邻来投票。这叫做**懒惰学习（Lazy Learning）**。

Q5-01 KNN 工作原理：手动计算分类

题目描述：

训练集有以下 6 个数据点（2 个特征），标签为 A 类或 B 类：

编号	x_1	x_2	类别
1	1	1	A
2	2	2	A
3	3	1	A
4	6	5	B
5	7	4	B
6	5	6	B

新样本为 $Q = (4, 3)$，使用 KNN 算法（欧氏距离）进行分类。

问题：

(1) 计算 Q 到所有 6 个训练点的欧氏距离（保留两位小数），填写下表：

训练点	坐标	到 $Q(4,3)$ 的距离	类别
1	(1,1)	__	A
2	(2,2)	__	A
3	(3,1)	__	A
4	(6,5)	__	B
5	(7,4)	__	B
6	(5,6)	__	B

(2) 当 $k = 3$ 时，找出 3 个最近邻，并通过投票确定 Q 的预测类别。

(3) 当 $k = 5$ 时，预测类别是否改变？

(4) KNN 用于**回归**时，预测方式与分类有何不同？设 6 个点的目标值分别为 $[10, 12, 11, 30, 28, 31]$，$k = 3$ 时 Q 的预测值是多少？

来源溯源：[文档第 5 章 5.1 KNN 算法介绍]

参考答案与详细解析

(1) 距离计算（$d = \sqrt{(x_1 - 4)^2 + (x_2 - 3)^2}$）：

训练点	坐标	距离	类别
1	(1,1)	$\sqrt{9+4}=\sqrt{13}\approx 3.61$	A
2	(2,2)	$\sqrt{4+1}=\sqrt{5}\approx 2.24$	A
3	(3,1)	$\sqrt{1+4}=\sqrt{5}\approx 2.24$	A
4	(6,5)	$\sqrt{4+4}=\sqrt{8}\approx 2.83$	B
5	(7,4)	$\sqrt{9+1}=\sqrt{10}\approx 3.16$	B
6	(5,6)	$\sqrt{1+9}=\sqrt{10}\approx 3.16$	B

(2) $k=3$：

最近 3 个点（按距离排序）：点 2（2.24，A），点 3（2.24，A），点 4（2.83，B）。投票：A 得 2 票，B 得 1 票。**预测为 A 类**。

(3) $k=5$：

最近 5 个点：点 2（A），点 3（A），点 4（B），点 5（B），点 6（B）。投票：A 得 2 票，B 得 3 票。**预测变为 B 类**——k 的选择对结果有重要影响。

(4) KNN 回归：

KNN 回归预测新样本的输出为 k 个最近邻目标值的**平均值**（而非投票）。$k=3$ 时最近 3 个点为点 2（目标值 12）、点 3（11）、点 4（30）：

$$\hat{y}=\frac{12+11+30}{3}=\frac{53}{3}\approx \mathbf{17.67}$$

Q5-02 k 值的影响：过大/过小的后果

题目描述：

KNN 中最关键的超参数是 k（近邻数），它直接控制模型复杂度。

问题：

(1) 当 $k=1$ 时，KNN 的决策边界是什么形状？它的训练误差是多少？为什么？

(2) 当 $k=n$（n 为训练集样本总数）时，KNN 的预测结果是什么？这对应欠拟合还是过拟合？

(3) 随着 k 从 1 增大到 n，模型的偏差和方差分别如何变化？（描述趋势）

(4) 在实际使用中，通常如何选择最优的 k？写出用 sklearn 的 `GridSearchCV` 进行 k 值网格搜索的核心代码。

(5) KNN 的时间复杂度是多少（预测一个新样本时）？当训练集很大时，有什么加速策略？

来源溯源：[文档第 5 章 5.1 关键参数 / 5.2 API 使用]

参考答案与详细解析

(1) $k=1$ **时：**

决策边界是 Voronoi 图（每个训练点控制其周围最近的区域），边界极度不规则（锯齿状）。训练误差**精确为 0**——因为每个训练点的最近邻就是它自己，预测结果与真实标签完全一致。这是典型的过拟合：决策边界高度拟合训练数据的噪声，泛化能力差。

(2) $k = n$ 时：

所有训练样本都是近邻，预测结果是训练集中**多数类别**（对分类）或**所有训练样本目标值的均值**（对回归）——对所有新样本一视同仁，完全忽略输入特征，**对应欠拟合**（高偏差，低方差）。

(3) 偏差-方差随 k 变化：

k 从 1 增大到 n：偏差**单调增大**（平滑化导致决策边界越来越粗略）；方差**单调减小**（平均更多邻居，对单个噪声点不敏感）。最优 k 在偏差-方差权衡的" 最低总误差" 处。

(4) 网格搜索选 k：

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV

param_grid = {'n_neighbors': range(1, 31, 2)}  # 搜索奇数k，避免平票
knn = KNeighborsClassifier()
grid_search = GridSearchCV(knn, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_scaled, y_train)
print(f"最优k: {grid_search.best_params_}")
print(f"最优CV准确率: {grid_search.best_score_:.3f}")

```

(5) 时间复杂度与加速：

朴素 KNN 预测一个新样本需要计算它与所有 n 个训练样本的距离，复杂度为 $O(nd)$（d 为维度），总预测复杂度 $O(nmd)$（m 个测试样本）。训练集大时极慢。

KNN 加速数据结构

KD-Tree（K-Dimensional Tree）：将训练数据组织为二叉树，预测时只搜索邻近分支，平均复杂度 $O(d \log n)$，但高维时退化。

Ball-Tree：用超球体组织数据，对高维数据比 KD-Tree 更鲁棒。

sklearn 的 `KNeighborsClassifier(algorithm='auto')` 会自动选择合适的数据结构。

Q5-03 距离度量：欧氏、曼哈顿、闵可夫斯基

题目描述：

KNN 的核心操作是计算距离，距离度量的选择影响" 近邻" 的定义，从而影响分类结果。

题目数据：两个点 $P = (1, 2, 3)$，$Q = (4, 0, 6)$（三维空间）。

问题：

(1) 计算 P 和 Q 之间的：

a. **欧氏距离**（L_2 范数）：$d = \sqrt{\sum_i (p_i - q_i)^2}$

b. **曼哈顿距离**（L_1 范数）：$d = \sum_i |p_i - q_i|$

c. **切比雪夫距离**（L_∞ 范数）：$d = \max_i |p_i - q_i|$

(2) 闵可夫斯基距离（Minkowski Distance）的公式是 $d = (\sum_i |p_i - q_i|^r)^{1/r}$，当 $r = 1, 2, \infty$ 时分别对应上面哪种距离？

(3) 填写下表，对比三种距离的特点与适用场景：

距离度量	几何直觉	适用场景
欧氏距离	_____	特征无明显方向偏好；连续数值特征
曼哈顿距离	_____	_____
切比雪夫距离	_____	_____

(4) 在 KNN 中使用距离度量时，为什么**必须先对特征进行归一化或标准化**？用具体数字说明不标准化的危害。

来源溯源：[文档第 5 章 5.3 常见距离度量方法]

参考答案与详细解析

(1) 距离计算（$P = (1, 2, 3), Q = (4, 0, 6)$，差值为 $[3, -2, 3]$）：

a. **欧氏距离：** $\sqrt{3^2 + (-2)^2 + 3^2} = \sqrt{9 + 4 + 9} = \sqrt{22} \approx \mathbf{4.69}$

b. **曼哈顿距离：** $|3| + |-2| + |3| = 3 + 2 + 3 = \mathbf{8}$

c. **切比雪夫距离：** $\max(3, 2, 3) = \mathbf{3}$

(2) 闵可夫斯基的统一框架：

$r = 1$：曼哈顿距离；$r = 2$：欧氏距离；$r \to \infty$：切比雪夫距离（极限情况）。

(3) 特点与适用场景：

距离度量	几何直觉	适用场景
欧氏距离	两点之间的直线距离（空中飞行）	连续特征，无明显方向性；图像像素距离
曼哈顿距离	只能沿轴走的出租车距离（城市街道）	特征独立性强，稀疏高维数据（如文本 TF-IDF）
切比雪夫距离	棋盘上国王走一步能到的范围	当最大单维差异是主要关注点时；棋类 AI

(4) 不标准化的危害（具体数字示例）：

假设特征 1 是年龄（20–60，范围 40），特征 2 是年收入（20000–200000，范围 180000）。不标准化时，两点 $A = (30, 50000)$ 和 $B = (50, 50000)$ 的欧氏距离 $= \sqrt{400 + 0} = 20$；而 $A = (30, 50000)$ 和 $C = (30, 51000)$ 的距离 $= \sqrt{0 + 1000000} = 1000$。年收入变化 1000 元的距离竟是年龄变化 20 岁的 50 倍——年龄特征**完全被淹没**，KNN 的”近邻”完全由年收入决定，年龄对结果毫无影响，这与实际业务含义严重不符。

Q5-04 KNN API 与完整案例：心脏病预测

题目描述：

下面是使用 sklearn KNN 进行心脏病预测的部分代码框架，请结合代码回答问题。

```python
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score
import joblib

# 1. 加载数据
df = pd.read_csv('heart_disease.csv')
X = df.drop('target', axis=1)
y = df['target']

# 2. 数据集划分
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 3. 特征工程
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)   # 注意：只transform，不fit

# 4. 模型训练
knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn.fit(X_train_scaled, y_train)

# 5. 评估
y_pred = knn.predict(X_test_scaled)
print(classification_report(y_test, y_pred))

# 6. 超参数调优
param_grid = {'n_neighbors': range(1, 21), 'weights': ['uniform', 'distance']}
grid_search = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)
grid_search.fit(X_train_scaled, y_train)

# 7. 保存模型
joblib.dump(grid_search.best_estimator_, 'knn_model.pkl')

```

问题：

(1) 解释第 3 步中，为什么测试集只能用 `transform` 而不能用 `fit_transform`？

(2) `KNeighborsClassifier` 中 `metric='minkowski'`, `p=2` 等价于哪种距离度量？如果改为 `p=1` 呢？

(3) `weights='distance'` 和 `weights='uniform'`7有什么区别？各在什么情况下更好？

(4) 第 6 步的网格搜索中，搜索空间一共有多少种参数组合？使用 5 折交叉验证，总共需要训练多少

参考答案与详细解析

(1) 只 transform 测试集的原因：

`fit_transform` 会**重新计算**均值和标准差，测试集有不同的统计量，用测试集统计量缩放后的数据与训练集缩放后的数据**不在同一尺度空间**，KNN 计算的距离会失去意义（就像训练时用公制、预测时用英制）。正确做法是用训练集的统计量（`scaler` 已记住）对测试集做相同的缩放。

(2) 距离等价：

`metric='minkowski'`，`p=2` 等价于**欧氏距离**（L_2）；`p=1` 等价于**曼哈顿距离**（L_1）。

(3) weights 参数：

`uniform`（默认）：所有近邻**等权重**投票，简单民主；`distance`：近邻权重与距离**成反比**（越近权重越高），如 $w_i = 1/d_i$。`distance` 在边界样本较多时更准确（近的邻居更有参考价值）；若数据噪声较多，`uniform` 更稳健（避免单个非常近的噪声点主导）。

(4) 搜索空间：

$n_neighbors$ 有 20 个取值，$weights$ 有 2 个取值，总共 $20 \times 2 = 40$ 种参数组合。5 折交叉验证每组参数训练 5 个模型，共训练 $40 \times 5 = \mathbf{200}$ 个模型。

(5) 模型保存与预测：

`joblib.dump` 将训练好的模型对象序列化保存到磁盘，`joblib.load` 加载回来，无需重新训练：

```python
import joblib
import numpy as np

# 加载模型和scaler（两个都要保存！）
model = joblib.load('knn_model.pkl')
scaler = joblib.load('scaler.pkl')  # 记得也要保存scaler

# 预测新样本（必须用同一个scaler处理）
x_new = np.array([[63, 1, 3, 145, 233, 1, 0, 150, 0, 2.3, 0, 0, 1]])
x_new_scaled = scaler.transform(x_new)
pred = model.predict(x_new_scaled)
pred_proba = model.predict_proba(x_new_scaled)
print(f"预测类别：{pred[0]}，患病概率：{pred_proba[0][1]:.3f}")

```

工程提醒：scaler 和 model 必须配套保存

很多初学者只保存模型忘记保存 scaler——部署时用了不同的缩放参数，预测结果会完全错误。实际项目中，`scaler`、`label encoder`、`model` 等所有预处理对象都要保存。

现实应用场景

KNN 的实际使用场景与局限：

KNN 在小规模数据（几千到几万条）、特征工程充分的场景下表现良好，常用于推荐系统的 item-based 协同过滤、异常检测（距所有训练点都很远的点是异常）、以及作为快速验证基线。其主要局限：预测慢（需遍历训练集）；高维失效（维度诅咒：高维空间中距离失去意义）；内存占用大（需存储全部训练集）。这些局限使其在工业大规模场景中较少单独使用。

2 本章小结

- **KNN 核心思想：**懒惰学习，预测时找 k 个最近邻投票/取均值——简单但计算代价高。
- **k 值权衡：**k 小 $\rightarrow$ 低偏差高方差（过拟合）；k 大 $\rightarrow$ 高偏差低方差（欠拟合）；用交叉验证选最优 k。
- **距离度量要选对：**欧氏距离最常用；曼哈顿适合高维稀疏；特征缩放是使用任何距离度量的前提。
- **必须标准化：**KNN 基于距离，量纲不一致会导致某个特征完全主导结果，其他特征失效。
- **模型保存：**scaler 和 model 必须配套保存，部署时用同一套预处理参数。